

DATA ANALYTICS

FASE 3 | AULA 04 -

LINGUAGEM SQL PARA ANALISE, SELEÇÃO, FILGRAM, AGRUPAMENTO E JOINS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	9
MERCADO, CASES E TENDÊNCIAS	14
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS	17

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, vamos explorar dois poderosos conjuntos de ferramentas de manipulação de dados: SQL e PySpark. O objetivo é fornecer uma compreensão sólida de como realizar operações de seleção, filtragem, agrupamento e joins em ambos os ambientes. Vamos abordar os conceitos teóricos, seguidos de exemplos e exercícios, para que você possa aplicar o que aprendeu na prática.

Além disso, discutiremos as principais categorias de comandos SQL: DDL (Data Definition Language), DML (Data Manipulation Language), DCL (Data Control Language) e TCL (Transaction Control Language), que oferecem uma visão abrangente da interação com bancos de dados.

HANDS ON

Comparativo SQL e PySpark

Seleção de Dados

SQL: no SQL, a operação de seleção é realizada com o comando SELECT. Isso permite que você especifique quais colunas de uma tabela você deseja visualizar.

Exemplo:

```
SELECT nome, idade FROM clientes;
```

Neste exemplo, estamos selecionando as colunas "nome" e "idade" da tabela "clientes".

PySpark: no PySpark, a seleção de colunas é feita usando o método select(). O DataFrame é a estrutura de dados fundamental no PySpark, semelhante a uma tabela em SQL.

Exemplo:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Exemplo").getOrCreate()
df = spark.read.csv("clientes.csv", header=True, inferSchema=True)
df.select("nome", "idade").show()
```

Aqui, estamos criando uma sessão Spark, carregando um arquivo CSV em um DataFrame e selecionando as colunas "nome" e "idade".

Exercício

Selecione os nomes e cidades de todos os clientes em ambos os ambientes.

SQL:

```
SELECT nome, cidade FROM clientes;
```

PySpark:

```
df.select("nome", "cidade").show()
```

Filtragem de Dados

SQL: a filtragem de dados em SQL é realizada com a cláusula *WHERE*, que permite restringir os resultados a um subconjunto específico de dados.

Exemplo:

```
SELECT nome FROM clientes WHERE idade > 30;
```

Neste caso, estamos filtrando os clientes com idade superior a 30 anos.

PySpark: em PySpark, a filtragem é feita usando o método `filter()`. A sintaxe é um pouco diferente, mas o conceito é o mesmo.

Exemplo:

```
df.filter(df.idade > 30).select("nome").show()
```

Aqui, estamos filtrando o DataFrame para incluir apenas os clientes com idade superior a 30.

Exercício

Liste os clientes que moram em "São Paulo" e que têm idade superior a 25 anos.

SQL:

```
SELECT nome FROM clientes WHERE cidade = 'São Paulo' AND idade > 25;
```

PySpark:

```
df.filter((df.cidade == 'São Paulo') & (df.idade > 25)).select("nome").show()
```

Agrupamento

SQL: o agrupamento em SQL é feito com o comando *GROUP BY*, que permite agregar dados e calcular estatísticas, como contagens ou médias.

Exemplo:

```
SELECT cidade, COUNT(*) FROM clientes GROUP BY cidade;
```

Neste exemplo, estamos contando o número de clientes por cidade.

PySpark: em PySpark, usamos o método `groupBy()`, que é semelhante ao GROUP BY do SQL. Depois de agrupar, podemos aplicar funções de agregação.

Exemplo:

```
df.groupBy("cidade").count().show()
```

Aqui, estamos agrupando os dados por cidade e contando o número de registros em cada grupo.

Exercício

Calcule a média de idade dos clientes em cada cidade.

SQL:

```
SELECT cidade, AVG(idade) FROM clientes GROUP BY cidade;
```

PySpark:

```
df.groupBy("cidade").agg({"idade": "avg"}).show()
```

Filtragem de Grupos

SQL: para filtrar os resultados de um grupo, utilizamos a cláusula HAVING, que permite aplicar condições sobre os resultados agregados.

Exemplo:

```
SELECT cidade, COUNT(*) FROM clientes GROUP BY cidade HAVING  
COUNT(*) > 10;
```

Neste exemplo, estamos filtrando as cidades que têm mais de 10 clientes.

PySpark: no PySpark, aplicamos a filtragem após o agrupamento usando o método `filter()`.

Exemplo:

```
df.groupBy("cidade").count().filter(df["count"] > 10).show()
```

Aqui, estamos contando o número de clientes por cidade e filtrando para mostrar apenas aquelas com mais de 10.

Exercício

Liste as cidades com mais de 5 clientes.

SQL:

```
SELECT cidade, COUNT(*) FROM clientes GROUP BY cidade HAVING  
COUNT(*) > 5;
```

PySpark:

```
df.groupBy("cidade").count().filter(df["count"] > 5).show()
```

Joins

SQL: os joins em SQL são usados para combinar dados de duas ou mais tabelas com base em uma condição de correspondência. O tipo mais comum é o INNER JOIN, que retorna apenas as linhas que têm correspondência em ambas as tabelas.

Exemplo:

```
SELECT c.nome, p.data FROM clientes c INNER JOIN pedidos p ON c.id =  
p.cliente_id;
```

Aqui, estamos selecionando os nomes dos clientes e as datas de pedidos, onde existe uma correspondência entre clientes e pedidos.

PySpark: em PySpark, usamos o método `join()` para combinar DataFrames. A sintaxe é semelhante, mas o resultado é um novo DataFrame.

Exemplo:

```
df_clientes = spark.read.csv("clientes.csv", header=True, inferSchema=True)  
df_pedidos = spark.read.csv("pedidos.csv", header=True, inferSchema=True)  
  
df_clientes.join(df_pedidos, df_clientes.id == df_pedidos.cliente_id,  
"inner").select("nome", "data").show()
```

Neste exemplo, estamos fazendo um `INNER JOIN` entre os DataFrames de clientes e pedidos.

Exercício

Encontre todos os pedidos feitos por clientes.

SQL:

```
SELECT c.nome, p.data FROM clientes c LEFT JOIN pedidos p ON c.id =  
p.cliente_id;
```

PySpark:

```
df_clientes.join(df_pedidos, df_clientes.id == df_pedidos.cliente_id,  
"left").select("nome", "data").show()
```

SAIBA MAIS

Contexto das Funções

Seleção de Dados (SELECT):

A seleção de dados é uma operação fundamental em qualquer análise de dados. Ela permite que você escolha quais colunas e registros deseja visualizar em um conjunto maior de dados. Isso é crucial para focar nas informações relevantes para sua análise, evitando a sobrecarga de dados desnecessários.

- **SQL:** a instrução SELECT é usada para especificar as colunas que você deseja retornar. Você pode selecionar uma ou mais colunas, e também usar a cláusula DISTINCT para eliminar duplicatas.
- **PySpark:** o método select() é utilizado em um DataFrame para especificar quais colunas você deseja ver. Isso é similar ao SELECT do SQL, mas usa uma sintaxe baseada em objetos.

Filtragem de Dados

Filtrar dados permite que você restrinja os resultados a um subconjunto específico, baseando-se em condições. Isso é útil quando você deseja analisar apenas os dados que atendem a certos critérios, como encontrar clientes com idades acima de um determinado limite ou produtos que custam menos que um valor específico.

- **SQL:** a cláusula WHERE é utilizada para especificar as condições de filtragem. Você pode combinar várias condições usando operadores lógicos como AND, OR e NOT.
- **PySpark:** o método filter() permite aplicar condições semelhantes sobre o DataFrame, utilizando expressões booleanas.

Agrupamento de Dados

Agrupar dados é essencial para realizar análises estatísticas, como calcular somas, médias ou contagens. Essa operação é frequentemente usada em relatórios e dashboards para fornecer resumos e insights a partir de grandes conjuntos de dados.

- **SQL:** a cláusula GROUP BY agrupa registros que têm valores iguais em colunas específicas. Após agrupar, você pode usar funções agregadas como COUNT(), SUM(), AVG(), entre outras.
- **PySpark:** o método groupBy() é usado para criar grupos no DataFrame. Após o agrupamento, você pode aplicar funções de agregação usando o método agg().

Filtragem de Grupos

Após agrupar dados, pode ser necessário filtrar os resultados agregados. Isso permite que você refine suas análises e foque apenas nos grupos que são significativos para suas necessidades de negócio ou pesquisa.

- **SQL:** a cláusula HAVING é usada para aplicar condições em resultados agregados, funcionando de forma semelhante à cláusula WHERE, mas para dados que já foram agrupados.
- **PySpark:** após agrupar, você pode usar filter() ou where() para refinar ainda mais os resultados, aplicando condições sobre os dados agregados.

Joins

Os joins são usados para combinar dados de duas ou mais tabelas com base em uma condição de correspondência. Essa operação é crucial para análises que exigem dados de múltiplas fontes, permitindo uma visão mais completa e integrada.

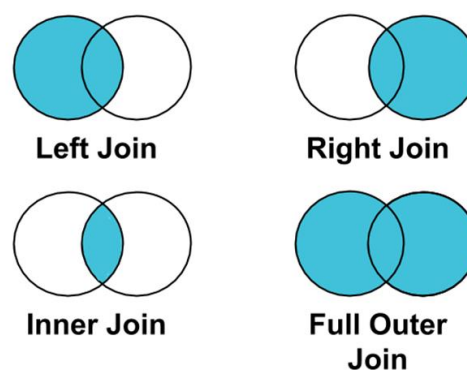


Figura 1 – Joins

Fonte: <https://pt.linkedin.com/pulse/como-funcionam-os-joins-jair-paiva-neves-pgwvpf> (2025)

Tipos de Joins

INNER JOIN: o INNER JOIN retorna apenas as linhas que têm correspondência em ambas as tabelas. É o tipo de join mais comum, utilizado quando você deseja ver apenas os dados que estão presentes em ambas as fontes.

Exemplo SQL:

```
SELECT c.nome, p.data  
FROM clientes c  
INNER JOIN pedidos p ON c.id = p.cliente_id;
```

Este exemplo retornará apenas os clientes que têm pedidos registrados.

Exemplo PySpark:

```
df_clientes.join(df_pedidos, df_clientes.id == df_pedidos.cliente_id,  
"inner").select("nome", "data").show()
```

LEFT JOIN (ou LEFT OUTER JOIN): o LEFT JOIN retorna todas as linhas da tabela à esquerda e as linhas correspondentes da tabela à direita. Se não houver correspondência, o resultado ainda incluirá as linhas da tabela à esquerda, mas com valores nulos nas colunas da tabela à direita.

Exemplo SQL:

```
SELECT c.nome, p.data  
FROM clientes c  
LEFT JOIN pedidos p ON c.id = p.cliente_id;
```

Este exemplo retorna todos os clientes, incluindo aqueles que não têm pedidos, com NULL nas colunas de pedidos.

Exemplo PySpark:

```
df_clientes.join(df_pedidos, df_clientes.id == df_pedidos.cliente_id,  
"left").select("nome", "data").show()
```

RIGHT JOIN (ou RIGHT OUTER JOIN): o RIGHT JOIN é o oposto do LEFT JOIN. Ele retorna todas as linhas da tabela à direita e as linhas correspondentes da tabela à esquerda. Se não houver correspondência, as linhas da tabela à direita ainda aparecem, com valores nulos nas colunas da tabela à esquerda.

Exemplo SQL:

```
SELECT c.nome, p.data  
FROM clientes c  
RIGHT JOIN pedidos p ON c.id = p.cliente_id;
```

Aqui, todos os pedidos são retornados, incluindo aqueles que não têm clientes associados.

Exemplo PySpark:

```
df_clientes.join(df_pedidos, df_clientes.id == df_pedidos.cliente_id,  
"right").select("nome", "data").show()
```

FULL JOIN (ou FULL OUTER JOIN): o FULL JOIN retorna todas as linhas de ambas as tabelas, combinando onde há correspondência. Se não houver correspondência, as linhas serão retornadas com valores nulos nas colunas da tabela que não tiver correspondência.

Exemplo SQL:

```
SELECT c.nome, p.data  
FROM clientes c  
FULL JOIN pedidos p ON c.id = p.cliente_id;
```

Isso retorna todos os clientes e todos os pedidos, independentemente de haver ou não correspondência.

Exemplo PySpark:

```
df_clientes.join(df_pedidos, df_clientes.id == df_pedidos.cliente_id,  
"full").select("nome", "data").show()
```

CROSS JOIN: o CROSS JOIN retorna o produto cartesiano das duas tabelas, ou seja, combina cada linha da primeira tabela com cada linha da segunda. Esse tipo de join pode resultar em um número muito grande de linhas, especialmente se ambas as tabelas forem grandes.

Exemplo SQL:

```
SELECT c.nome, p.data  
FROM clientes c  
CROSS JOIN pedidos p;
```

Este exemplo combina todos os clientes com todos os pedidos.

Exemplo PySpark:

```
df_clientes.crossJoin(df_pedidos).select("nome", "data").show()
```

Entender como utilizar as diferentes funções e tipos de joins é fundamental para manipular e analisar dados de maneira eficaz. Cada tipo de join tem sua própria finalidade, e a escolha correta pode proporcionar insights valiosos a partir de seus dados. A combinação de SQL e PySpark oferece uma flexibilidade incrível para realizar essas operações, seja em ambientes de dados relacionais ou em grandes conjuntos de dados distribuídos. Se você precisar de mais detalhes ou exemplos específicos, sinta-se à vontade para perguntar. Estamos sempre à disposição no Discord!

MERCADO, CASES E TENDÊNCIAS

O Mercado de Dados

A demanda por profissionais que dominam **SQL** e **PySpark** está em alta devido ao crescimento exponencial do volume de dados gerados diariamente. Empresas dos setores de **finanças**, **e-commerce**, **saúde** e **tecnologia** buscam especialistas capazes de lidar com dados de forma eficiente, impulsionando decisões baseadas em insights. As posições mais requisitadas incluem analista de dados, engenheiro de dados, cientista de dados e engenheiro de Big Data.

Salários para essas posições refletem essa alta demanda, com profissionais experientes em SQL e PySpark frequentemente ganhando acima da média do mercado. Plataformas de empregos como LinkedIn e Glassdoor mostram um aumento consistente na publicação de vagas que exigem conhecimentos em ambas as ferramentas.

Cases de Sucesso

Amazon: utiliza PySpark para processar grandes volumes de dados relacionados a recomendações de produtos, permitindo análises em tempo real para personalização da experiência do cliente.

Netflix: integra PySpark para analisar padrões de consumo de mídia, otimizando algoritmos de recomendação com base em comportamentos de usuários.

Uber: emprega SQL para monitorar operações locais em tempo real e PySpark para análises complexas, como otimização de rotas e previsão de demandas.

Spotify: combina SQL e PySpark para criar relatórios analíticos e prever tendências musicais, aprimorando recomendações e playlists personalizadas.

Setor Bancário: bancos utilizam SQL para controle transacional e PySpark para detectar fraudes em larga escala, analisando padrões em milhões de transações por segundo.

Tendências em SQL e PySpark

Integração com tecnologias de Big Data: ferramentas como Apache Spark, Hadoop, Delta Lake e Google BigQuery têm se integrado cada vez mais com o SQL e PySpark, permitindo que as empresas aproveitem melhor seus dados distribuídos.

Arquiteturas em Nuvem: com a popularização de plataformas como AWS Glue, Databricks e Azure Synapse, o uso de PySpark combinado com SQL tornou-se ainda mais acessível. Isso possibilita análises escaláveis e descentralizadas.

Data Lakehouse: o modelo de Data Lakehouse, que combina as melhores características de Data Lakes e Data Warehouses, vem ganhando força. SQL é usado para acessar dados estruturados, enquanto PySpark é empregado para dados não estruturados e semiestruturados.

Automação com Machine Learning: SQL e PySpark estão sendo usados como base para pipelines de Machine Learning, permitindo automação de análises avançadas e suporte à decisão em tempo real.

Popularização de ETL Moderno: ferramentas de ETL como Airflow, Apache NiFi e dbt frequentemente integram SQL e PySpark, promovendo fluxos de dados eficientes e otimizados.

Low-Code e No-Code: plataformas como Alteryx e Tableau estão facilitando o uso de SQL para profissionais não técnicos, enquanto PySpark continua sendo usado em segundo plano para operações mais complexas.

Desafios do Mercado

Apesar da alta demanda, empresas enfrentam desafios como:

- **Escassez de talentos qualificados:** muitos profissionais ainda não têm experiência prática em ambientes de Big Data e ferramentas distribuídas.
- **Evolução constante das ferramentas:** ferramentas como PySpark evoluem rapidamente, exigindo aprendizado contínuo.
- **Complexidade de integração:** trabalhar com dados distribuídos e ferramentas híbridas (SQL + PySpark) requer um alto nível de coordenação entre equipes de engenharia e dados.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, revisamos conceitos de seleção, filtragem, agrupamento e joins, tanto em SQL quanto em PySpark. Aprendemos como executar operações semelhantes em ambos os ambientes, permitindo que você escolha a ferramenta certa para a tarefa certa.

Aplicações Práticas do SQL e PySpark: a prática em SQL e PySpark permite que você analise dados de maneira eficiente, seja em bancos de dados relacionais ou em grandes conjuntos de dados distribuídos. Cada ferramenta tem suas próprias vantagens e desvantagens, e a escolha entre elas depende do tipo de dados e da análise que você deseja realizar.

Dominar SQL e PySpark é essencial para qualquer profissional na área de dados. Com essas habilidades, você estará preparado e preparada para enfrentar desafios analíticos em diversas indústrias. A combinação de conhecimento em SQL e PySpark permitirá que você se destaque no mercado de trabalho e contribua de maneira significativa para sua organização.

REFERÊNCIAS

CARVALHO, E. F. **SQL: Guia Prático para Iniciantes**. São Paulo: Novatec, 2019.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. Rio de Janeiro: Elsevier, 2004.

FERNANDES, L. M. **Aprendendo SQL em 24 Horas**. São Paulo: Alta Books, 2018.

MAURO. **A Diferença Entre INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL OUTER JOIN**. **BLOG DO SQL**. [s.d]. Disponível em: <https://blogdosql.com.br/a-diferenca-entre-inner-join-left-join-right-join-e-full-outer-join/>. Acesso em 26 mar. 2025.

MENDES, F. J. **Introdução ao SQL: Como Começar a Manipular Bancos de Dados Relacionais**. Revista Brasileira de Computação, v. 10, n. 2, 2022, p. 35-50.

MYSQL. **Documentação Oficial do MySQL: Introdução ao SQL**. Oracle. 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 26 mar. 2025.

NIEDERAUER, C. R. **Banco de Dados com SQL: Introdução à Linguagem e Práticas**. Rio de Janeiro: Ciência Moderna, 2020.

PINTO, R. S.; ARAÚJO, T. L. **SQL Básico e Avançado: Fundamentos Essenciais para a Prática**. Florianópolis: Visual Books, 2021.

POSTGRESQL. **Iniciando com SQL no PostgreSQL**. PostgreSQL Documentation, 2024. Disponível em: <https://www.postgresql.org/docs/current/sql.html>. Acesso em: 26 mar. 2025.

SQLITE. **Guia de Introdução ao SQLite**. SQLite Documentation. 2024. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 26 mar. 2025.

PALAVRAS-CHAVE

Bancos de Dados. SQL. Modelagem de Dados.

EXEMPLO

The background is a dark blue field filled with numerous small, light blue dots. Overlaid on this are several large, wavy, translucent lines in shades of blue and yellow. A vertical line with a small circle at the top and an 'x' at the bottom is positioned on the left. A circle containing the number '7' is located in the upper center. A hexagon is visible in the lower right corner.

POSTECH